

Руководство по Python

Совет: Это руководство предназначено для *программистов*, которые только начинают изучать язык Python, **а не новичков**, которые только знакомятся с программированием.

Python — это простой в освоении, но мощный язык программирования. Он обладает эффективными высокоуровневыми структурами данных и простым, но действенным подходом к объектно-ориентированному программированию. Элегантный синтаксис и динамическая типизация Python, а также интерпретируемый характер языка делают его идеальным для написания скриптов и быстрой разработки приложений во многих областях на большинстве платформ.

Интерпретатор Python и обширная стандартная библиотека находятся в свободном доступе в виде исходного кода или бинарных файлов для всех основных платформ на сайте Python, <https://www.python.org/>, и могут распространяться бесплатно. На том же сайте можно найти дистрибутивы и ссылки на множество бесплатных сторонних модулей, программ и инструментов на Python, а также дополнительную документацию.

Интерпретатор Python легко дополняется новыми функциями и типами данных, реализованными на C или C++ (или других языках, вызываемых из C). Python также подходит в качестве языка расширения для настраиваемых приложений.

В этом руководстве читатель в неформальной манере знакомится с основными понятиями и функциями языка и системы Python. Имейте в виду, что предполагается, что у вас есть базовые знания в области программирования в целом. Для практического опыта полезно иметь под рукой интерпретатор Python, но все примеры самодостаточны, поэтому руководство можно читать и без подключения к интернету.

Описание стандартных объектов и модулей см. в [Стандартной библиотеке Python](#). [Справочник по языку Python](#) содержит более формальное определение языка. Чтобы написать расширения на C или C++, прочтите [Расширение и встраивание интерпретатора Python](#) и [Справочник по Python/C API](#). Кроме того, существует несколько книг, подробно посвященных Python.

Это руководство не претендует на полноту и не охватывает все без исключения или даже все часто используемые возможности языка. Вместо этого оно знакомит вас со многими наиболее примечательными особенностями Python и дает хорошее представление о характере и стиле языка. Прочитав его, вы научитесь читать и писать модули и программы на Python и будете готовы узнать больше о различных модулях библиотеки Python, описанных в [«Стандартной библиотеке Python»](#).

Также стоит ознакомиться с [гlossарием](#).

- [1. Разжигаем аппетит](#)
- [2. Использование интерпретатора Python](#)
 - [2.1. Запуск интерпретатора](#)
 - [2.1.1. Передача аргументов](#)
 - [2.1.2. Интерактивный режим](#)
 - [2.2. Интерпретатор и его окружение](#)

- 2.2.1. Кодировка исходного кода
- 3. Неформальное введение в Python
 - 3.1. Использование Python в качестве калькулятора
 - 3.1.1. Числа
 - 3.1.2. Текст
 - 3.1.3. Списки
 - 3.2. Первые шаги в программировании
- 4. Дополнительные инструменты управления потоком
 - 4.1. if Высказывания
 - 4.2. for Высказывания
 - 4.3. range() Функция
 - 4.4. break и continue операторы
 - 4.5. else Предложения о циклах
 - 4.6. pass Операторы
 - 4.7. match Операторы
 - 4.8. Определение функций
 - 4.9. Подробнее об определении функций
 - 4.9.1. Значения аргументов по умолчанию
 - 4.9.2. Аргументы ключевых слов
 - 4.9.3. Специальные параметры
 - 4.9.3.1. Аргументы с указанием позиции или ключевого слова
 - 4.9.3.2. Параметры только с указанием позиции
 - 4.9.3.3. Аргументы только с указанием ключевого слова
 - 4.9.3.4. Примеры функций
 - 4.9.3.5. Краткое описание
 - 4.9.4. Произвольные списки аргументов
 - 4.9.5. Распаковка списков аргументов
 - 4.9.6. Лямбда-выражения
 - 4.9.7. Строки документации
 - 4.9.8. Аннотации функций
 - 4.10. Интермеццо: стиль программирования
- 5. Структуры данных
 - 5.1. Подробнее о списках
 - 5.1.1. Использование списков в качестве стеков
 - 5.1.2. Использование списков в качестве очередей
 - 5.1.3. Генераторы списков
 - 5.1.4. Генераторы вложенных списков
 - 5.2. Оператор del
 - 5.3. Кортежи и последовательности
 - 5.4. Множества
 - 5.5. Словари
 - 5.6. Методы циклического перебора
 - 5.7. Подробнее об условиях
 - 5.8. Сравнение последовательностей и других типов
- 6. Модули
 - 6.1. Подробнее о модулях

- 6.1.1. Выполнение модулей как скриптов
- 6.1.2. Путь поиска модулей
- 6.1.3. «Скомпилированные» файлы Python
- 6.2. Стандартные модули
- 6.3. Функция `dir()`
- 6.4. Пакеты
 - 6.4.1. Импорт `*` из пакета
 - 6.4.2. Ссылки внутри пакета
 - 6.4.3. Пакеты в нескольких каталогах
- 7. Ввод и вывод
 - 7.1. Более сложное форматирование вывода
 - 7.1.1. Форматированные строковые литералы
 - 7.1.2. Метод `String format()`
 - 7.1.3. Ручное форматирование строк
 - 7.1.4. Старое форматирование строк
 - 7.2. Чтение и запись файлов
 - 7.2.1. Методы файловых объектов
 - 7.2.2. Сохранение структурированных данных с помощью `json`
- 8. Ошибки и исключения
 - 8.1. Синтаксические ошибки
 - 8.2. Исключения
 - 8.3. Обработка исключений
 - 8.4. Генерация исключений
 - 8.5. Цепочка исключений
 - 8.6. Пользовательские исключения
 - 8.7. Определение действий очистки
 - 8.8. Предусмотренные действия очистки
 - 8.9. Генерация и обработка нескольких несвязанных исключений
 - 8.10. Добавление примечаний к исключениям
- 9. Классы
 - 9.1. Немного об именах и объектах
 - 9.2. Области видимости и пространства имён в Python
 - 9.2.1. Пример областей видимости и пространств имён
 - 9.3. Первый взгляд на классы
 - 9.3.1. Синтаксис определения класса
 - 9.3.2. Объекты класса
 - 9.3.3. Объекты экземпляра
 - 9.3.4. Объекты метода
 - 9.3.5. Переменные класса и экземпляра
 - 9.4. Random Remarks
 - 9.5. Inheritance
 - 9.5.1. Multiple Inheritance
 - 9.6. Private Variables
 - 9.7. Odds and Ends
 - 9.8. Iterators
 - 9.9. Generators

- 9.10. Generator Expressions
- 10. Brief tour of the standard library
 - 10.1. Operating system interface
 - 10.2. File wildcards
 - 10.3. Command-line arguments
 - 10.4. Error output redirection and program termination
 - 10.5. String pattern matching
 - 10.6. Mathematics
 - 10.7. Internet access
 - 10.8. Dates and times
 - 10.9. Data compression
 - 10.10. Performance measurement
 - 10.11. Quality control
 - 10.12. Batteries included
- 11. Brief tour of the standard library — part II
 - 11.1. Output formatting
 - 11.2. Templating
 - 11.3. Working with binary data record layouts
 - 11.4. Multi-threading
 - 11.5. Logging
 - 11.6. Weak references
 - 11.7. Tools for working with lists
 - 11.8. Decimal floating-point arithmetic
- 12. Virtual Environments and Packages
 - 12.1. Introduction
 - 12.2. Creating Virtual Environments
 - 12.3. Managing Packages with pip
- 13. What Now?
- 14. Interactive Input Editing and History Substitution
 - 14.1. Tab Completion and History Editing
 - 14.2. Alternatives to the Interactive Interpreter
- 15. Floating-Point Arithmetic: Issues and Limitations
 - 15.1. Representation Error
- 16. Appendix
 - 16.1. Interactive Mode
 - 16.1.1. Error Handling
 - 16.1.2. Executable Python Scripts
 - 16.1.3. The Interactive Startup File
 - 16.1.4. The Customization Modules